NANYANG TECHNOLOGICAL UNIVERSITY

EE6407 Genetic Algorithms & Machine Learning

Assignment 1 & 2 Report

MSc in Electrical and Electronic Engineering
Yan Ziming
G2507084J

October 14, 2025

Contents

1 Introduction

For the course EE6407, I implement two assignments, including the Naïve Bayes Classifier and the Fisher Linear Discriminate Classifier. This report summarizes the methods and results of these two assignments. I use Python as the programming language and Jupyter Notebook as the developement environment. The followings show the details of my design and implementation.

2 Assignment 1

2.1 Handling Missing Values and Outliers

2.1.1 Missing values

Detection:

For the missing value detection, I use the pandas library to check, with isna() function. This function would return a matrix reflects if the content in each position is empty or not. By summing the values in each column, I can determine the number of times each feature's missing values occur 1.

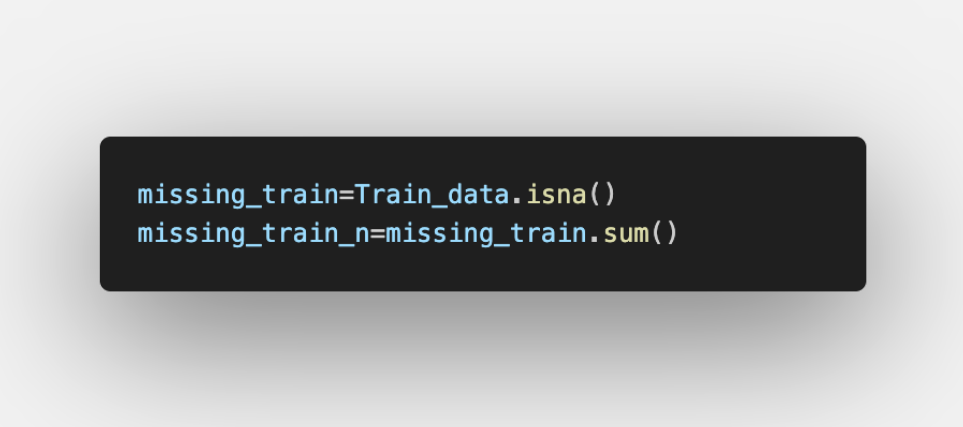

Figure 1: Missing Value Detection Code

.

At first, there seems no missing values. however, after trying to transform datat type from dtype to float, there seems several data cannot successfully transformed, leading to new missing values. The missing value detection for training set consequence 1:

Feature	Missing Value Count
A	2
B	0
C	0
D	1
E	0

Table 1: Missing Value Count

Handling:

Since there are only several missing values in each feature, ig. two for feature A and one for feature D. I think that there are three methods could be used to replace the empty with practical values [1]:

- *Missing Deletion:*

  Simply delete all the feature at the position of missing value. This method is simple and effective, but may lead to data loss and bias of training classifier. In this condition, since the total number of missing values is 3, which is 2.5% of the total data, I think this method may be acceptable.

- *Single Imputation:*

  Replace the missing value with a specific value, such as mean, median, or k nearest neighbors. This method avoid the data loss, and consider the distribution of data. However, when there are plenty of missing value, this method may replace these values with same data, leading to bias.

- *Imputation with Machine Learning:*

  Use machine learning methods, regression, to predict the missing value. For example, use linear regression, decision tree regression, or least square regression to predict. Compared with single imputation, this method is more reliable. But it is more complex and time-consuming.

In this assignment, I use the mean value of each feature to replace the missing values. Because the number of missing values is small, I think there is no need to use more complex methods, like machine learning. And this implementation is more practical than deletion 2.

Figure 2: Imputation of Missing Values

2.1.2 Outliers

Detection:

As for the outlier data, which means the data that is far away from the main distribution of dataset and may affect the performance of classifier. There are two mainstream methods to detect outliers, IQR and Z score. Because the dataset is small and may not follow Gaussian distribution. Therefore, I used the IQR method to detect, which calculates the IQR value with the first and third quartile (Q_1,Q_3), regarding values that out of the range [$Q_1 - 1.5 \times IQR$,

$Q_3 + 1.5 \times IQR$] as outliers. I used the following code 3 to implement the number of outliers in each feature 2. The training dataset has 5 outliers in total, while there is no outlier in test dataset.

```
def IQR(data):
    Q1=data.quantile(.25)
    Q3=data.quantile(.75)
    IQR=Q3-Q1
    outlier=(data<(Q1-1.5*IQR))|(data>(Q3+
1.5*IQR))
    return outlier

outlier_train_IQR=IQR(Train_data)
outlier_test_IQR=IQR(Test_data)
outlier_train_n=outlier_train_IQR.sum()
outlier_test_n=outlier_test_IQR.sum()
```

Figure 3: IQR Method Realization

Feature	Outlier Count
A	1
B	2
C	1
D	1
E	0

Table 2: Outlier Count

In order to show the context of outliers, I visualized the distribution of each feature with boxplot 4.

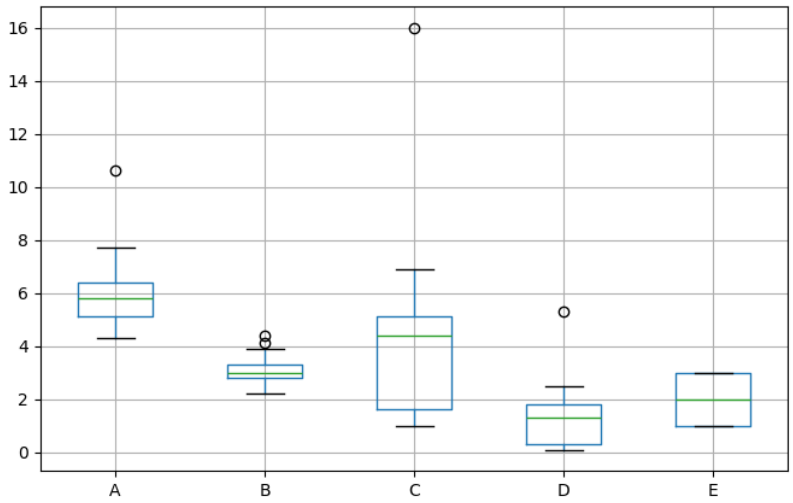

Figure 4: Boxplot of Training Set

Handling:

Similar to the missing value, I come up with three methods may be used to handle the outliers.

- *Outlier Removeal:*

  This is the simplest and most direct method, just remove the outlier data with all features. This is effective, but may lead to data loss and training errors.

- *Imputation:*

  Replace the outlier data with a specific value, such as mean and median. If there are many outliers, this method may lead to too many same values and affect the training of classifiers.

- *Capping Method:*

  As mentioned before, the outliers are the values that out of IQR range. This method replace the too high value with the value of 95th percentile, and replace the too low value with the value of 5th percentile. This method can meep the distribution, but may have same demerits as imputation with many outliers.

In this assignment, I use the capping method to handle the outliers 5, and visualize the dataset with boxplot after cleaning 6.

```
Train_capped = Train_data.copy()
for col in Train_data.columns:
    Train_capped[col] = np.where(
    Train_data[col] < lower[col], p05[col],
    np.where(
        Train_data[col] > upper[col], p95[
col],
        Train_data[col])
    )
```

Figure 5: Capping Realization

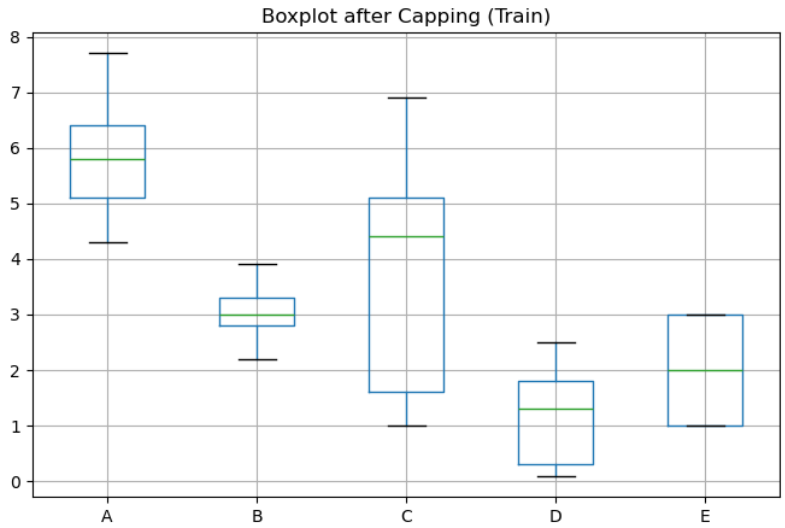

Figure 6: Boxplot of Training Set After Cleaning

2.2 Design a Naïve Bayes Classifier

A Naïve Bayes Classifier is a simple classification model based on Bayes' Theorem with one important assumption: each feature is independent with each other (idd). I will introduce the establishment of classifier in the following parts.

What we want to compute for prediction is $P(C_k|\mathbf{x})$. According to Bayes' Theorem,

$$P(C_k \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid C_k)\, P(C_k)}{P(\mathbf{x})} \tag{1}$$

The $\mathbf{x}$ has many features:[x_1, x_2, x_3, ..., x_n]. Since the assumption of idd, we can simplify the $P(\mathbf{x} \mid C_k)$:

$$P(\mathbf{x} \mid C_k) = P(x_1, x_2, x_3, ..., x_n \mid C_k) = \prod_{i=1}^{n} P(x_i \mid C_k) \tag{2}$$

Therefore,

$$P(C_k \mid \mathbf{x}) = \frac{P(C_k) \prod_{i=1}^{n} P(x_i \mid C_k)}{P(\mathbf{x})} \tag{3}$$

According to maximum a posterior (MAP) estimation,

$$\hat{C} = \arg\max_{C_k} \frac{P(C_k) \prod_{i=1}^{n} P(x_i \mid C_k)}{P(\mathbf{x})} \tag{4}$$

For all classes, $P(\mathbf{x})$ is constant. Therefore, we can ignore it and use log function to avoid underflow:

$$\hat{C} = \arg\max_{C_k} \left(\ln P(C_k) + \sum_{i=1}^{n} \ln P(x_i \mid C_k) \right) \tag{5}$$

The (5) shows the decision function (rule) of Naive Bayes Classifier. $P(C_k)$ could be estimate by the ratio of class C_k in training set.

As for the $P(x_i \mid C_k)$, since the data is continuous and not binary, we can assume that they follow Gaussian distribution. With the Maximum Likelihood Estimation (MLE), we can estimate the mean and variance of each feature in each class. Therefore,

$$P(x_i \mid C_k) = \frac{1}{\sqrt{2\pi\sigma_{ik}^2}} \exp\left(-\frac{(x_i - \mu_{ik})^2}{2\sigma_{ik}^2} \right) \tag{6}$$

$$\mu_{ik} = \frac{1}{N_k} \sum_{x \in C_k} x_i \tag{7}$$

$$\sigma_{ik}^2 = \frac{1}{N_k} \sum_{x \in C_k} (x_i - \mu_{ik})^2 \tag{8}$$

With the above equations, I successfully establish a Gaussian Naive Bayes Classifier, the coding realization is shown 7.

```
class GNB_Classifier:
    def __init__(self):
        pass

    def fit(self,X,y):
        self.classes=np.unique(y)
        self.means=[]
        self.vars=[]
        self.priors=[]
        for w in self.classes:
            X_w=X[y==w]
            prior_w=X_w.shape[0]/X.shape[0]
            mean_w=np.mean(X_w,axis=0)

# cov_w=np.cov(X_w,rowvar=False)+1e-6*np.eye
(X.shape[1])
            var_w = np.var(X_w, axis=0)
            self.priors.append(prior_w)
            self.means.append(mean_w)
            # self.vars.append(cov_w)
            self.vars.append(var_w)

        self.priors = np.array(self.priors)
        self.means = np.array(self.means)
        self.vars = np.array(self.vars)


    def ConditionPwithGaussian(self,x,mean,
var):
        eps = 1e-6
        coeff = 1.0 / np.sqrt(2 * np.pi *
var + eps)
        exponent = -  x - mean) ** 2 /  2 *
var + eps)          (                (
        return coeff * np.exp(exponent)
```

```
def predict(self, X):
    X = np.array(X)
    y_pred = []

    for x in X:
        posteriors = []
        for i, c in enumerate(self.classes):
            prior_log = np.log(self.priors[i
])
            likelihood_log = np.sum(np.log(
self.ConditionPwithGaussian(x, self.means[i
], self.vars[i])))
            posteriors.append(prior_log +
likelihood_log)
        y_pred.append(self.classes[np.argmax
(posteriors)])

    return np.array(y_pred)

def evaluate(self, X, y):
    y = np.array(y)
    y_pred = self.predict(X)
    accuracy = np.mean(y_pred == y)
    return accuracy
```

Figure 7: Code of Classifier Establishment

2.3 Prediction and Analysis

After the development of classifier, I trained the model with training dataset, A-D for features and E for label. Then I predict the test dataset labels. In addition, I tested the accuracy with training dataset and it seems pretty good. The consequences are shown below.

Table 3: Results of Gaussian Naive Bayes Classifier

Metric	Value
Training Accuracy	0.958333
Predicted Labels	[1 1 1 1 1 1 1 1 1 1 3 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3]

3 Assignment 2

3.1 Fisher Linear Discriminate Classifier

3.1.1 Establishment of FLD

The Fisher LInear Discriminate (FLD) is a classifier to classify datasets which only have two classes. The main idea of FLD is to project the data into a line and maximize the distance between two classes while minimize the variance in each class. The following parts introduce the establishment roughly. Namely, we want to find a projection vector $\mathbf{w}$, which can maximize the following function:

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S_B} \mathbf{w}}{\mathbf{w}^T \mathbf{S_W} \mathbf{w}} \tag{9}$$

(**i**) *Get the label of two classes.*

For the unknown dataset, the first thing is to get the labels of two classes 8. After coding, the labels of two classes are [-1, 1].

```
y=np.array(label_train[
'label_train'])
np.unique(y)
```

Figure 8: Get Classes Code

(**ii**) *Find the distance between two classes.*

The distance between two classes is the distance between the mean of each class. Therefore, I need to calculate the mean of each class first **??**, then calculate the distance **??**.

$$\mathbf{m_k} = \frac{1}{N_k} \sum_{x \in C_k} \mathbf{x} \tag{10}$$

$$\mathbf{d} = \mathbf{m_1} - \mathbf{m_2} \tag{11}$$

(**iii**) *Calculate the within-class scatter matrix.*

3.1.2 Training and Parameters

After setting up the FLD classifier, I trained the model with training dataset and got the weights vector **w** and bias b with the function inside class code. The parameters are shown below.

Table 4: Results of Fisher Linear Discriminant Analysis (LDA)

Parameter	Value
w	[-0.00178602 -0.0034729 -0.00194673 -0.00168257 -0.00265808]
b	0.008700

3.2 Prediction and Analysis

Table 5: Results of Fisher Linear Discriminant Analysis (LDA)

Metric	Value
Predicted Labels	[-1 -1 -1 -1 -1 -1 -1 -1 -1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
Training Accuracy	0.932409

4 Conclusion

References

[1] L. Ren, T. Wang, A. Sekhari Seklouli, H. Zhang, and A. Bouras, “A review on missing values for main challenges and methods,” *Information Systems*, vol. 119, p. 102268, Oct. 2023.